

OBJECT-ORIENTED PROGRAMMING (C#)

Course Assignment: Object Relationships & Implementation

Student

Name:

Date:

Student ID:

Section:

Mandatory Technical Requirements

- Every class you create **MUST** strictly follow the encapsulation principles.
- Each class must include:
 - **Private Field(s)** (properly named using camelCase or with an underscore prefix).
 - **Public Property(ies)** (with standard getters and setters to access the fields).
 - **Constructor(s)** to properly initialize class fields.
 - A public function named `PrintInfo()` where the explicit return type is `string`.

Assignment Tasks

Task 1: Inheritance (Is-a Relationship)

Implement a foundational class hierarchy to demonstrate the concept of specialization through inheritance.

- Create a base class named `Person` that encapsulates a person's name.
- Create a derived class named `Student` that inherits from the `Person` class and introduces a specific `StudentId` property.
- **Requirement:** Ensure that the `PrintInfo()` method in the `Student` class overrides or extends the base behavior to return a formatted string containing both the student's name and their unique ID.

Task 2: Aggregation (Has-a Relationship - Weak Bond)

Model a structural relationship where one object acts as a container for another independent object.

- Create a `Professor` class that encapsulates the professor's name.
- Create a `Department` class that contains a department name and a reference to a `Professor` object who manages it.
- **Requirement:** Establish an **Aggregation** relationship. The professor must exist independently of the department and must be passed as an external reference to the department's constructor.
- The `PrintInfo()` method in the `Department` class must return a complete string comprising both the department's name and the managing professor's name.

Task 3: Composition (Part-of Relationship - Strong Bond)

Design a tight lifecycle relationship where the contained object cannot exist without the owner object.

- Create a `Room` class that contains a specific room number.
- Create a `House` class that contains a physical address property and an internal `Room` object.
- **Requirement:** Implement a strict **Composition** relationship. The `Room` object's lifecycle must be tied directly to the `House`. It must be instantiated *inside* the house's constructor.
- The `House` class's `PrintInfo()` method must retrieve the internal room's information and return a combined string detailing both the house address and its room number.

Task 4: Association (Uses-a Relationship)

Demonstrate a loose, temporary interaction between two completely autonomous objects.

- Create a `Patient` class and a `Doctor` class, each encapsulating their respective names independently.
- **Requirement:** Establish an **Association** relationship. Implement an additional distinct method inside the `Doctor` class named `Checkup()`.
- The `Checkup()` method must accept a `Patient` object as a parameter and return a `string` explicitly stating that Doctor [Name] is currently examining Patient [Name].

Task 5: Execution & Verification (Main Method)

Verify the structural correctness and integration of your classes within the main application entry point.

- In the `Program` class (inside the standard `Main` method), instantiate sample objects for all the tasks defined above.
- Utilize `Console.WriteLine()` to print out the final `string` outputs returned by the respective `PrintInfo()` and `Checkup()` methods.
- Ensure the application compiles without errors and successfully demonstrates all four distinct OOP relationships.